# Лекция 3. Суффиксный массив, суффиксное дерево.

## 1. Введение

Строка – последовательность символов из алфавита Σ. **Суффикс** строки s длины n – это подстрока, начинающаяся с позиции i и идущая до конца: s[i..n-1]. **Префикс** – подстрока от начала до некоторой позиции. Суффиксные структуры данных (массив и дерево) позволяют эффективно решать множество задач, связанных с подстроками: поиск вхождений, подсчёт количества различных подстрок, поиск наибольшей общей подстроки, лексикографическое сравнение суффиксов и т.д.

## 2. Суффиксный массив

**Суффиксный массив** (suffix array) строки s – это массив целых чисел sa[0..n-1], содержащий начальные позиции всех суффиксов строки s, упорядоченных лексикографически. Для удобства к строке часто добавляют терминальный символ $, который меньше любого символа алфавита – это гарантирует, что ни один суффикс не является префиксом другого.

### 2.1 Пример

Пусть s = "abacaba$". Выпишем все суффиксы:

0: abacaba$
1: bacaba$
2: acaba$
3: caba$
4: aba$
5: ba$
6: a$
7: $

Отсортировав их лексикографически, получим суффиксный массив:

| **индекс суффикса** | **суффикс** |
|---|---|
| 7 | $ |
| 6 | a$ |
| 4 | aba$ |
| 0 | abacaba$ |
| 2 | acaba$ |
| 5 | ba$ |
| 1 | bacaba$ |
| 3 | caba$ |

Итак, sa = [7, 6, 4, 0, 2, 5, 1, 3].

### 2.2 Простейший алгоритм построения

Можно просто взять все суффиксы, представить их как строки и отсортировать стандартной сортировкой. Однако сравнение двух суффиксов в худшем случае занимает O(n), поэтому общая сложность будет $O(n^2 \cdot \log n)$. Это неприемлемо для длинных строк.

### 2.3 Алгоритм построения за $O(n \cdot \log n)$ (метод удвоения)

Идея: на каждом шаге сортируем суффиксы по первым $2^k$ символам, используя результаты предыдущего шага. Пусть на шаге k мы имеем для каждой позиции i ранг rank[i] – порядковый номер суффикса s[i..] при сортировке по первым $2^k$ символам. Для шага k+1 нам нужно отсортировать по первым $2^{k+1}$ символам. Для этого представим каждый суффикс парой (rank[i], $rank[i + 2^k]$) (если $i + 2^k$ выходит за границу, считаем ранг равным -1, т.к. терминальный символ меньше всех). Тогда сортировка этих пар даст новый порядок.

Сортировку пар можно выполнить цифровой сортировкой за O(n) по второму элементу, затем стабильно по первому. Или использовать сортировку подсчётом, если размер алфавита рангов не превышает n.

**Псевдокод построения суффиксного массива:**

```
function build_suffix_array(s):
  n = length(s)
  // Инициализация: сортируем по одному символу
  // создаём массив sa и массив рангов
  sa = array of indices 0..n-1
  sort sa by s[i]   // можно сортировкой подсчётом, если алфавит небольшой
  rank = array of size n
  rank[sa[0]] = 0
  for i from 1 to n-1:
    rank[sa[i]] = rank[sa[i-1]] + (s[sa[i]] != s[sa[i-1]])

  k = 1
  while k < n:
    // готовим пары для сортировки: (rank[i], rank[i+k])
    // используем сортировку подсчётом по второму ключу, затем по первому
    // временные массивы
    temp_sa = array of size n
    // сортировка по второму ключу (rank[i+k])
    cnt = array of size n filled 0
    for i in 0..n-1:
      second = (i + k < n) ? rank[i + k] : -1
```

```
        cnt[second + 1]++   // смещение для -1
    // преобразуем cnt в префиксные суммы
    for i from 1 to n:
        cnt[i] += cnt[i-1]
    // заполняем temp_sa, идя справа налево для стабильности
    for i from n-1 down to 0:
        second = (sa[i] + k < n) ? rank[sa[i] + k] : -1
        cnt[second + 1]--
        temp_sa[cnt[second + 1]] = sa[i]
    // сортировка по первому ключу (rank[i])
    cnt = array of size n filled 0
    for i in 0..n-1:
        first = rank[temp_sa[i]]
        cnt[first]++
    for i from 1 to n-1:
        cnt[i] += cnt[i-1]
    for i from n-1 down to 0:
        first = rank[temp_sa[i]]
        cnt[first]--
        sa[cnt[first]] = temp_sa[i]
    // вычисляем новые ранги
    new_rank = array of size n
    new_rank[sa[0]] = 0
    for i from 1 to n-1:
        cur = sa[i]
        prev = sa[i-1]
        cur_first = rank[cur]
        cur_second = (cur + k < n) ? rank[cur + k] : -1
        prev_first = rank[prev]
        prev_second = (prev + k < n) ? rank[prev + k] : -1
        new_rank[cur] = new_rank[prev] + ( (cur_first != prev_first) or (cur_second
!= prev_second) )
    rank = new_rank
    k *= 2
  return sa
```

Время работы: $O(n \cdot \log n)$ за счёт логарифмического числа итераций и O(n) на сортировку подсчётом на каждой итерации.

### 2.4 Вычисление LCP (наибольшего общего префикса)

Часто вместе с суффиксным массивом используется массив LCP (Longest Common Prefix) – для каждой пары соседних суффиксов в sa хранится длина их общего префикса. Существует эффективный алгоритм Касаи (Kasai) за O(n).

**Псевдокод:**

```
function build_lcp(s, sa):
   n = length(s)
   rank = array of size n   // обратный массив: rank[ позиция ] = индекс в sa
   for i in 0..n-1:
      rank[sa[i]] = i
   lcp = array of size n-1 filled 0
   k = 0
   for i in 0..n-1:   // i – позиция в исходной строке (суффикс s[i:])
      if rank[i] == n-1:
         k = 0
         continue
      j = sa[rank[i] + 1]   // следующий суффикс в порядке sa
      // увеличиваем k, пока символы совпадают
      while i + k < n and j + k < n and s[i + k] == s[j + k]:
         k++
      lcp[rank[i]] = k
      if k > 0:
         k--
   return lcp
```

Идея: при переходе к следующему суффиксу (сдвиг на 1 влево) LCP может уменьшиться не более чем на 1, поэтому общее число сравнений O(n).

### 2.5 Поиск подстроки с помощью суффиксного массива

Имея суффиксный массив, можно найти все вхождения образца P в текст s за $O(|P| \cdot \log n)$ с помощью бинарного поиска. Сравнение образца с суффиксом s[sa[mid]:] на каждой итерации бинарного поиска требует $O(|P|)$ времени, но можно ускорить, используя LCP для более быстрого сравнения.

Альтернативно, можно за $O(|P| + \log n)$ найти интервал суффиксов, начинающихся с P, выполняя два бинарных поиска (левую и правую границу) с использованием предвычисленных LCP.

## 3. Суффиксное дерево

Суффиксное дерево (suffix tree) строки s – это компактное представление всех суффиксов в виде дерева, где:

- каждый путь от корня до листа соответствует некоторому суффиксу;
- каждое ребро помечено непустой подстрокой s;
- внутренние узлы имеют не менее двух детей (кроме корня);
- листья соответствуют окончаниям суффиксов (обычно помечаются позицией начала суффикса).

Добавление терминального символа $ гарантирует, что никакой суффикс не является префиксом другого, поэтому листья однозначно соответствуют суффиксам.

### 3.1 Свойства

- Размер дерева (число узлов) не превосходит 2n-1, где n – длина строки с терминатором.
- Память линейна относительно длины строки.
- Все подстроки s могут быть найдены как пути от корня.
- Поиск подстроки P выполняется за $O(|P|)$ (спуск по дереву).

### 3.2 Построение: алгоритм Укконена

Алгоритм Укконена (Ukkonen) строит суффиксное дерево за O(n), добавляя символы строки по одному слева направо и поддерживая структуру. Он использует понятия **активной точки** (active point) и **суффиксных ссылок** (suffix links).

**Основные понятия:**

- **Активная точка** – текущая позиция в дереве, где будет происходить следующее добавление. Она задаётся узлом, ребром и смещением на ребре.
- **Суффиксная ссылка** из узла v (соответствующего строке α) ведёт в узел, соответствующий наибольшему собственному суффиксу α (т.е. отбрасыванию первого символа). Суффиксные ссылки позволяют быстро переходить между суффиксами.

**Идея алгоритма:**

На каждом шаге (добавление символа c) мы должны добавить c ко всем суффиксам текущей строки. Это делается путём прохода по суффиксам от самого длинного к самому короткому с помощью суффиксных ссылок. Различают три случая:

1. **Суффикс уже оканчивается в листе** – тогда просто удлиняем ребро, ведущее в лист, добавляя символ.
2. **Суффикс заканчивается внутри ребра, и следующего символа нет** – тогда нужно создать новый узел, разделив ребро, и добавить лист для нового суффикса.
3. **Суффикс заканчивается в узле или внутри ребра, и переход по c уже существует** – тогда можно просто переместить активную точку по этому переходу и завершить фазу (правило 3 останавливает дальнейший обход).

Алгоритм хранит **активную точку** (узел, ребро, длина) и для каждого внутреннего узла – суффиксную ссылку.

**Псевдокод (упрощённый):**

```
class Node:
  next: map char -> (Node, start, end)  // переходы по символам (куда, откуда индекс начала, индекс конца)
```

```
    link: Node  // суффиксная ссылка

function build_suffix_tree(s):
    n = length(s)
    root = new Node()
    // добавляем терминальный символ в конец (пусть будет $)
    // активная точка: сначала корень
    active_node = root
    active_len = 0
    active_edge = -1  // не используется, если активная точка в узле
    remainder = 0        // количество суффиксов, которые нужно добавить на
текущем шаге

    for i from 0 to n-1:
        char = s[i]
        // добавляем символ ко всем суффиксам
        remainder++
        last_created_node = None
        while remainder > 0:
            if active_len == 0:
                // активная точка в узле
                if char not in active_node.next:
                    // создаём новый лист
                    active_node.next[char] = (new Node(), i, INF)
                    // устанавливаем суффиксную ссылку для предыдущего созданного
узла, если есть
                    if last_created_node:
                        last_created_node.link = active_node
                        last_created_node = None
                else:
                    // переход есть – двигаемся по нему
                    next_node, start, end = active_node.next[char]
                    // переходим в следующий узел, но активная длина станет 1
                    // (на самом деле здесь более сложная логика, упрощаем)
                    active_node = next_node
                    active_len = 1
                    active_edge = start
                    // правило 3: если суффикс уже есть, прерываем цикл
                    // но для простоты опустим детали
                    break
            else:
                // активная точка на ребре
                # пропустим детали реализации
```

```
        # ...
        remainder--
    return root
```

Полная реализация алгоритма Укконена довольно объёмна, но ключевая идея – использование суффиксных ссылок для перехода между суффиксами и поддержание активной точки для эффективного добавления. В результате все операции амортизированно линейны.

### 3.3 Поиск подстроки в суффиксном дереве

Для поиска подстроки P нужно просто пройти по дереву, следуя символам P. Если на каком-то шаге нужного перехода нет – подстрока отсутствует. Если удалось пройти полностью – подстрока есть в тексте, а листья под полученным узлом укажут все позиции вхождений.

Время поиска – $O(|P|)$.

## 4. Сравнение суффиксного массива и суффиксного дерева

| **Характеристика** | **Суффиксный массив** | **Суффиксное дерево** |
|---|---|---|
| Память | ~ 4n байт (массив + LCP) | ~ 20n байт (узлы, рёбра, ссылки) |
| Построение | $O(n \cdot \log n)$ (сортировка), часто быстрее на практике | O(n) (алгоритм Укконена), но сложнее в реализации |
| Поиск подстроки | $O(\log n)$ (бинарный поиск) | O(P) |
| Доп. возможности | Проще хранить и передавать, удобен для внешней памяти | Естественная поддержка многих задач (повторы, палиндромы и т.д.) |

На практике выбор зависит от задачи и ограничений по памяти/времени. Суффиксный массив часто предпочтительнее благодаря меньшему потреблению памяти и относительной простоте реализации, особенно с использованием эффективных алгоритмов построения. Суффиксное дерево мощнее для сложных запросов, но требует аккуратной реализации.